Homework 7: Verification, SMT, and Synthesis

17-355/17-665/17-819: Program Analysis
Fraser Brown and Yiliang Liang

Due: Thursday, November 19, 2026 11:59 pm EDT
(except for Question 3 – see instructions)

100 points
(+ 3% course grade bonus for Question 3)

Assignment Objectives:

- Reason about satisfiability and EUF theory and implement a solution in SMT-LIB format.
- Demonstrate understanding of specifications for verifying programs.
- Develop an SMT formulation and implement the solution for an Inductive Synthesis problem.
- *(Optional)* Read some papers about finding bugs in the real world.

Handin Instructions. This assignment has two components (Written and Coding). Questions 1 and 2a are part of the Written component, and Question 2b is part of the Coding component. The written component should be submitted on Gradescope as a file containing your answers (just like the written assignments before), and the coding component should be submitted as a GitHub repository to Gradescope. Question 3 is an optional question. See the instructions in the assignment for details.

Note: *We will not look at your homework repository directly, but will only see what you have submitted to Gradescope!* ***Make sure that you (re)submit after you have completed all parts; Gradescope does not automatically pull new commits from GitHub.***

Question 1, SMT with EUF (Written), *(25 points).*
a) (10 points) Show, using the congruence closure, whether the following Equality Logic with Uninterpreted Functions (EUF) formula is satisfiable. Show each step merging equivalent terms.

$$f(g(0)) = g(f(0)) \land f(g(f(y))) = 0 \land f(y) = 0 \land g(f(0)) \neq 0$$

b) (15 points) Give the SMT-LIB formula (i.e., a valid Z3 program) to prove your answer to (a) is correct. Put your code in the answer, and show the output of Z3.

Question 2, Inductive Synthesis (Written and Coding), *(75 points).*

This task asks you to write a simple synthesizer for an expression that satisfies a collection of input/output pairs. The instruction set consists of only four binary operators for bitvectors: `multiply`, `add`, `left bit shift`, and `bitwise or`. Your expression has four inputs `A`, `B`, `C`, `D`. Your task is to discover the right operators using the inputs such that it evaluates to the output.

Example. Suppose we have the input A = 2, B = 2, C = 1, D = 1, and the output 4. Two solutions are possible, and represented with different trees ((A + B) * (C * D)) and (A * (B * (C * D))).

More generally, we want to discover the operators and expression tree such that the inputs evaluate to the output.

a) (25 points) (Written) Develop a strategy to encode the general problem with SMT.

b) (50 points) (Coding) Make a private fork of the starter repository at

`https://github.com/CMU-program-analysis-fa26/homework-7`.

Follow the instructions in the repository and implement your solution (using your SMT formulation) to synthesize the expression satisfying the input/output pairs using Z3. The input/output pairs are provided in the starter repository. Inputs correspond to the format (A, B, C, D).

You are free to use any programming language that interfaces with Z3. APIs exist for C, Java, Python, OCaml, etc. See here. We provide starter code using Python in the provided repository.

Description. You may assume that each input is used only *once* in the expression, and there are four inputs. I.e., the solution is guaranteed to use each of the inputs A, B, C, and D, and will only use each of these values once. All values (inputs and outputs) are 16-bits wide. Note that inputs can occur in any order (`B << A` is a possible subexpression).

Test. Check your answer from your Z3 solution by making the `test.py` file (in the same directory) pass. That is: translate your synthesized expression into Python and add it to the test function. If you choose not to use Python, write a function like `test.py` that tests your answer against all the input/output pairs, and include instructions to run your solution in your `answers.pdf` writeup.

Guidelines. Permutations of binary trees correspond to the set of possible expression trees that can be generated. Leaf nodes correspond to inputs, and we know there are four leaf nodes since we have four inputs. Non-leaf nodes correspond to operators. Can you think of a way to encode permutations of binary trees with the operators in a way that Z3 can tell you which combination of operators satisfy the output?

Consider using boolean variables that correspond to operators which toggle whether an operator is used or not in an expression. If you set up the constraints the right way, Z3 can tell you which operators should be used to satisfy the solution (toggled "on") and which ones should be left out. This way, the Z3 solution consists of flags that correspond to the right operator for satisfying the input/output relation.

Question 3, Paper Reading on Bug-Finding, *(optional).*

Successful completion of this task may earn you extra credit, not to exceed an additional 3 percentage points for your course grade, at the discretion of the instructor(s). For example, if your course grade is 75% and you receive full credit for this task, your grade increases to 78%.

Throughout this class, we learned many program analysis and bug-finding techniques in a lot of theoretical details, but we never looked into how these techniques (and other methodologies derived from them) are used to find real bugs. This optional question asks you to read one or more research paper of your choosing from a list of pre-approved papers and topics, and prepare a short, 5-10 minutes class presentation summarizing the paper(s) you read.

Topic/Paper Selection. Please use this link to select a topic[1] by putting your name next to the topic that you choose. Each topic has one or more papers/tools associated with it. You do not need to read all of them, but you are certainly welcomed to read and discuss more than one.

If you have a topic in mind that is not listed (and some paper(s) in that topic that you would like to read), please discuss it with the instructor(s) for approval before proceeding.

Reading Paper(s). Read at least one paper / article associated with your chosen topic listed in the spreadsheet. You may read more if you wish. Pay particular attention to the problem being addressed, the methodology used, and the key results.

A few questions to consider while reading the paper(s) include:

- Why do previous approaches fail to address the problem effectively?
- What does this paper do differently to address the problem? What are the novel contributions? Why do these contributions work? *For example, just identifying that a paper implements fuzzing is not enough. What specific techniques of fuzzing did they use, and why did this technique work well for their specific problem domain? What properties of the system that they are analyzing make these techniques particularly applicable? How did they guide the fuzzing process? What did these techniques not achieve? These are all more insightful than "they used fuzzing."*
- What are the key findings? What bugs did they find using these techniques? Can you illustrate some of them?

Focus on the high-level ideas without getting bogged down by low-level technical and implementation details. A few of the topics also have some associated tools. You are welcomed, but not required, to explore and try out these tools.

Presentation. Summarize your findings into a short, 5-10 minute class presentation. You may (but are not required to) use slidedecks.

Deadlines (for Question 3). The deadline for this question differs from the other questions in this assignment.

- Topic selection: Tuesday, Dec. 1 at 11:59pm ET.
- Presentation: recitation section on Friday, Dec. 4, 10-10:50am ET.

No late days allowed for topic selection and presentation.

[1] Log in using your CMU email.

Acknowledgements. This assignment (and particularly the paper selection) is adapted from Rohan Padhye's course "Fantastic Bugs and How to Find Them" (17-712) which goes through these topics in much greater depth. It's a fun class, you should take it.